

TNVED.ai

Telegram-бот для операторов

Техническое задание | Sprint 3

Версия: 1.0

Длительность: 1 неделя (5 рабочих дней)

Предусловие: Sprint 1 + Sprint 2 завершены

Дата: 20 мая 2026

Заказчик: Бахитжан (Казахстан)

1. Цель спринта

Создать единый Telegram-бот @tnved_bot, через который 4-6 сотрудников TNVED.ai будут принимать packing list от китайских поставщиков и получать готовые таможенные инвойсы за минуты вместо часов.

Бот построен поверх API из Sprint 2 — никакой бизнес-логики в коде бота, только UX-слой (диалог, кнопки, сессии, аутентификация по Telegram ID).

1.1. Что должно работать в конце спринта

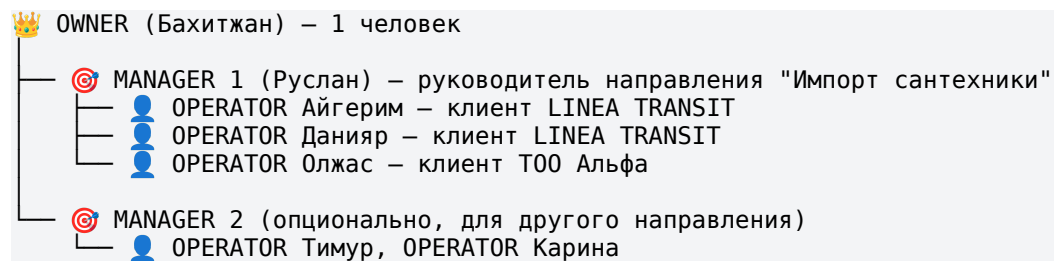
- Один бот @tnved_bot для всей команды
- 3 роли: OWNER, MANAGER, OPERATOR с разными правами
- OWNER (Бахитжан) может назначать MANAGER и видеть всё
- MANAGER управляет своей командой: добавляет/удаляет операторов, назначает им клиентов
- OPERATOR делает инвойсы по назначенным клиентам
- Полный flow: загрузка → классификация → ревью → одобрение → скачивание xlsx
- Переназначение задач, запрос помощи
- Логирование всех действий

1.2. Что НЕ войдёт в Sprint 3 (важно зафиксировать)

- ❌ Web-кабинет (это Sprint 4)
- ❌ Биллинг и KPI операторов (Sprint 5)
- ❌ Уведомления клиенту-импортёру (Sprint 5)
- ❌ Whitelabel-боты под отдельные компании (Sprint 5+)
- ❌ Двухфакторная аутентификация
- ❌ Многоязычность (только русский)

2. Ролевая модель

2.1. Иерархия



2.2. Матрица прав доступа

Действие	OWNER	MANAGER	OPERATOR
Создавать инвойс	✓	✓	✓
Видеть свои инвойсы	✓	✓	✓
Видеть инвойсы команды	✓ всех	✓ своих	✗
Назначать MANAGER	✓	✗	✗
Добавлять OPERATOR в команду	✓	✓ в свою	✗
Удалять пользователей	✓ любых	✓ из своей команды	✗
Переназначать задачи	✓	✓ в команде	✗
Запросить помощь у руководителя	—	✓ → OWNER	✓ → MANAGER
Управлять профилями клиентов	✓	✗	✗
Видеть статистику команды/компании	✓ всей компании	✓ своей команды	✓ только своя
Доступ к биллингу	✓	✗	✗
Резервные копии БД	✓	✗	✗

3. Команды бота

3.1. Общие команды (доступны всем после авторизации)

Команда	Действие
<code>/start</code>	Авторизация по telegram_user_id. Если в БД — приветствие. Если нет — отказ
<code>/menu</code>	Главное меню (кнопки в зависимости от роли)
<code>/new</code>	Создать новый инвойс (загрузить packing list)
<code>/list</code>	Список моих задач в работе
<code>/history</code>	История моих завершённых инвойсов
<code>/help</code>	Справка по командам
<code>/cancel</code>	Отменить текущий диалог
<code>/me</code>	Моя информация: роль, команда, доступные клиенты, статистика

3.2. Команды OPERATOR

Команда	Действие
<code>/clients</code>	Мои доступные клиенты
<code>/help_manager</code>	Запросить помощь у моего MANAGER по конкретному инвойсу

3.3. Команды MANAGER (дополнительно к operator)

Команда	Действие
<code>/team</code>	Управление командой (список операторов, +/-, переназначение)
<code>/team_list</code>	Все инвойсы команды
<code>/team_stats</code>	Статистика команды (KPI операторов)
<code>/add_operator</code>	Добавить нового оператора в команду
<code>/help_owner</code>	Запросить помощь у OWNER (Бахитжан)

3.4. Команды OWNER (дополнительно ко всему)

Команда	Действие
<code>/admin</code>	Админ-меню (полный контроль)
<code>/add_manager</code>	Назначить нового MANAGER
<code>/all_users</code>	Все пользователи системы (с ролями)

Команда	Действие
/all_invoices	Все инвойсы компании
/clients_admin	Управление клиентскими профилями
/billing	Финансовая сводка (выручка, расходы Claude API)
/backup	Запустить резервную копию БД
/broadcast	Отправить сообщение всем сотрудникам

4. Полные сценарии работы

4.1. Сценарий: первый запуск бота новым оператором

Айгерим (не в БД): /start
 Бот: ❌ Этот бот для сотрудников TNVED.ai.
 Если ты сотрудник — попроси своего руководителя
 добавить тебя по username @aigerim_dev

[Через 5 минут Руслан (MANAGER) добавил Айгерим]

Айгерим: /start
 Бот: 🙌 Добро пожаловать в TNVED.ai!
 Ты: Айгерим (роль: OPERATOR)
 Команда: Руслан (Импорт сантехники)
 Доступные клиенты: LINEA TRANSIT

Главное меню:
 [📦 Новый инвойс] [📋 Мои задачи] [📖 История]
 [👥 Мои клиенты] [❓ Помощь руководителя]

4.2. Сценарий: создание инвойса оператором

Айгерим: [📦 Новый инвойс]
 Бот: Выбери клиента:
 [LINEA TRANSIT]

Айгерим: [LINEA TRANSIT]
 Бот: Отправь packing list (xlsx, pdf или фото)

Айгерим: [пересылает 李小龙_1_发货明细.xlsx]
 Бот: ✅ Файл получен
 📊 67 позиций
 ⚖️ 31 176 кг брутто (предварительно)
 📦 213 670 шт

Режим стоимости?
 [💰 Целевые платежи \$] [⚖️ Цена за кг]
 [📋 Прайс клиента] [📊 Индикатив КГД]

Айгерим: [💰 Целевые платежи]
 Бот: Сколько \$? Например: 5000

Айгерим: 5000
 Бот: 🔄 Запускаю классификатор...
 ⏳ Это займёт 2-3 минуты

[Через 2 мин]
 Бот: ✅ Готово!

📊 Инвойс #2026-C351-0003
 Клиент: LINEA TRANSIT
 Позиций: 67 → 7 кодов ТН ВЭД

Брутто: 31 176 кг → Нетто: 29 618 кг
 Стоимость: \$20 875
 Пошлина: \$1 370
 НДС 16%: \$3 559
 Сбор: \$46

ВСЕГО ПЛАТЕЖЕЙ: \$4 975 (цель \$5 000) ✓

⚠️ 2 позиции требуют ревью:
 • #43: «金属配件 K48» (уверенность 65%)

- #58: фото нечёткое

[👁️ Посмотреть #43] [👁️ Посмотреть #58]
 [✅ ОДОБРИТЬ] [✎ РЕДАКТИРОВАТЬ] [❌ ОТМЕНИТЬ]

4.3. Сценарий: ревью спорной позиции

Айгерим: [👁️ Посмотреть #43]

Бот: Позиция #43:

Артикул: K48-12

Текст из файла: 金属配件 (металлические фитинги)

Кол-во: 1920 шт

Брутто: 384 кг

Фото: 📷 [фото товара]

Текущий код: 7412200000

Описание: Фитинги для труб из медных сплавов

Сставка пошлины: 3%

Уверенность LLM: 65% ▲

Альтернативы из БД:

1. 7412200000 (текущий) — Фитинги из медных сплавов
2. 7415310000 — Гайки, шурупы из меди
3. 8481808199 — Краны латунные

[✅ Оставить 1] [Выбрать 2] [Выбрать 3] [🔍 Свой код]

Айгерим: [✅ Оставить 1]

Бот: ✓ Подтверждено. Возвращаемся к инвойсу.

Осталось 1 позиция на ревью.

[👁️ Посмотреть #58]

4.4. Сценарий: одобрение и получение xlsx

Айгерим: [✅ ОДОБРИТЬ И СГЕНЕРИТЬ]

Бот: 🔄 Генерирую инвойс по шаблону LINEA TRANSIT...

[Через 5 секунд]

Бот: 📎 invoice_2026-C351-0003.xlsx (16 KB)

✓ Инвойс готов!

Время обработки: 3 мин 12 сек

[📧 Переслать клиенту] [📝 Новый инвойс]
 [📋 К моим задачам]

4.5. Сценарий: руководитель добавляет оператора

Руслан (MANAGER): /team

Бот: 👥 Твоя команда (3):

- Айгерим (LINEA TRANSIT) — активна
- Данияр (LINEA TRANSIT) — активен
- Олжас (T00 Альфа) — активен

[+ Добавить оператора] [📊 Статистика] [📋 Все задачи]

Руслан: [+ Добавить оператора]

Бот: Введи username нового оператора (например @ivan_dev):

Руслан: @karina_imp

Бот: Имя оператора (как обращаться):

Руслан: Карина

Бот: К каким клиентам у Карины доступ?
[LINEA TRANSIT] [T00 Альфа] [Все клиенты проекта]

Руслан: [LINEA TRANSIT]

Бот: ☒ Карина добавлена в твою команду.
Доступ: LINEA TRANSIT
Роль: OPERATOR
Передай ей команду /start

4.6. Сценарий: переназначение задачи

Руслан: /team_list

Бот: В работе у команды (5):
1. #0034 Айгерим — LINEA TRANSIT — 67 поз — 3 часа без активности
2. #0035 Данияр — LINEA TRANSIT — 42 поз — в работе
...

Руслан: [#0034]

Бот: Инвойс #0034 — LINEA TRANSIT
Оператор: Айгерим (3 часа без активности)
Статус: на ревью (2 позиции спорные)

[Открыть] [Переназначить] [Написать оператору]

Руслан: [Переназначить]

Бот: Кому передать?
[Данияр] [Олжас] [Я сам]

Руслан: [Данияр]

Бот: ☒ Инвойс #0034 переназначен на Данияра.
Айгерим уведомлена.
Данияр получил инвойс в свой список.

4.7. Сценарий: оператор просит помощи

Айгерим: /help_manager

Бот: По какому инвойсу нужна помощь?
[#0034] [#0035]

Айгерим: [#0034]

Бот: Опиши проблему (1-2 предложения):

Айгерим: Не уверена в коде для позиции #43,
фото нечёткое, в БД 3 варианта

Бот: ☒ Руслан уведомлён. Жди ответа.

[У Руслана появляется уведомление]

Бот → Руслан: Айгерим просит помощи
Инвойс: #0034 (LINEA TRANSIT)
Проблема: «Не уверена в коде для позиции #43,
фото нечёткое, в БД 3 варианта»

[Открыть инвойс] [Ответить Айгерим]

4.8. Сценарий: OWNER назначает MANAGER

Ты (OWNER): /admin

Бот: Админ-меню:
[Все пользователи] [Назначить MANAGER]
[Клиенты] [Статистика] [Биллинг]
[Настройки] [Резервная копия]

Ты: [Назначить MANAGER]

Бот: Username будущего руководителя:

Ты: @ruslan_chief

Бот: Имя:

Ты: Руслан

Бот: Название направления/проекта

(например: «Импорт сантехники из Китая»):

Ты: Импорт сантехники из Китая

Бот: ☒ Руслан назначен MANAGER

Проект: «Импорт сантехники из Китая»

Руслан получил уведомление.

Теперь он может добавлять операторов командой /add_operator

5. Расширение схемы БД (дополнения к Sprint 1+2)

5.1. Таблица telegram_users

```
CREATE TABLE telegram_users (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  telegram_user_id  BIGINT UNIQUE NOT NULL,      -- ID в Telegram
  telegram_username TEXT,                        -- @aigerim_dev
  full_name         TEXT NOT NULL,              -- "Айгерим Касымова"

  role              TEXT NOT NULL,              -- OWNER | MANAGER | OPERATOR
  team_name         TEXT,                      -- для MANAGER
  manager_id        UUID REFERENCES telegram_users(id), -- для OPERATOR

  client_access     JSONB DEFAULT '[]',         -- ["LINEA TRANSIT"] | ["*"]

  added_by          UUID REFERENCES telegram_users(id),
  added_at          TIMESTAMPTZ DEFAULT NOW(),
  last_active       TIMESTAMPTZ,
  is_active         BOOLEAN DEFAULT TRUE,

  language          TEXT DEFAULT 'ru',

  -- денормализованные счётчики для быстрых запросов
  invoices_total    INTEGER DEFAULT 0,
  invoices_this_month INTEGER DEFAULT 0,
  avg_confidence    NUMERIC(5,2),

  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_tguser_tg ON telegram_users(telegram_user_id);
CREATE INDEX idx_tguser_role ON telegram_users(role);
CREATE INDEX idx_tguser_manager ON telegram_users(manager_id);
CREATE INDEX idx_tguser_active ON telegram_users(is_active) WHERE is_active = TRUE;

CREATE UNIQUE INDEX idx_tguser_one_owner ON telegram_users(role)
  WHERE role = 'OWNER'; -- защита: только 1 OWNER
```

5.2. Таблица conversation_state (сессии бота)

Каждый пользователь может быть в одном из состояний диалога. Эта таблица хранит контекст между сообщениями.

```
CREATE TABLE conversation_state (
  telegram_user_id BIGINT PRIMARY KEY REFERENCES telegram_users(telegram_user_id),
  state            TEXT NOT NULL,              -- IDLE | UPLOADING | SELECTING_CLIENT | ...
  context          JSONB DEFAULT '{}',        -- временные данные (invoice_id, step, etc.)
  updated_at       TIMESTAMPTZ DEFAULT NOW()
);

-- Возможные состояния:
-- IDLE                  -- ждёт команды
-- AWAITING_FILE         -- ждёт packing list
-- AWAITING_CLIENT_CHOICE -- выбор клиента
-- AWAITING_PRICE_MODE   -- выбор режима стоимости
-- AWAITING_PRICE_VALUE  -- ввод суммы
-- PROCESSING            -- идёт классификация
-- REVIEWING             -- ревью результата
-- AWAITING_HELP_DESCRIPTION -- описание проблемы для MANAGER
```

```
-- ADDING_OPERATOR_USERNAME -- ввод username нового оператора
-- ADDING_OPERATOR_NAME     -- ввод имени нового оператора
-- ADDING_OPERATOR_CLIENTS  -- выбор клиентов для нового оператора
-- ... и т.д.
```

5.3. Таблица help_requests

```
CREATE TABLE help_requests (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  from_user_id  UUID REFERENCES telegram_users(id),
  to_user_id   UUID REFERENCES telegram_users(id),
  invoice_id   UUID REFERENCES invoices(id),
  description  TEXT NOT NULL,
  status       TEXT DEFAULT 'PENDING', -- PENDING | RESOLVED
  response     TEXT,
  created_at   TIMESTAMPTZ DEFAULT NOW(),
  resolved_at  TIMESTAMPTZ
);
```

5.4. Таблица audit_log (журнал действий)

```
CREATE TABLE audit_log (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_user_id  UUID REFERENCES telegram_users(id),
  action       TEXT NOT NULL, -- USER_ADDED | USER_REMOVED | INVOICE_APPROVED |
  ...
  target_type  TEXT,          -- user | invoice | client
  target_id   UUID,
  payload     JSONB,          -- детали действия
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_audit_actor ON audit_log(actor_user_id, created_at DESC);
CREATE INDEX idx_audit_target ON audit_log(target_type, target_id);
```

5.5. Расширение invoices (из Sprint 2)

```
-- Добавляем поля для Telegram-интеграции
ALTER TABLE invoices ADD COLUMN created_via TEXT DEFAULT 'web';
-- 'web' | 'telegram_bot'
ALTER TABLE invoices ADD COLUMN telegram_message_id BIGINT;
-- ID сообщения в Telegram для редактирования
ALTER TABLE invoices ADD COLUMN assigned_to UUID REFERENCES telegram_users(id);
-- кто работает над инвойсом сейчас
ALTER TABLE invoices ADD COLUMN reassigned_from UUID REFERENCES telegram_users(id);
-- если был переназначен
```

6. Архитектура бота

6.1. Стек технологий

- Node.js + TypeScript
- grammY (Telegram Bot framework для TS) — современная альтернатива node-telegram-bot-api
- Redis для FSM-состояний (хранение диалогов)
- BullMQ для долгих задач (классификация занимает 2-3 минуты)
- Подключается к Supabase напрямую и к API из Sprint 2 через HTTP

6.2. Структура проекта

```
tnved-bot/
├── src/
│   ├── handlers/                                # обработчики команд и кнопок
│   │   ├── common/
│   │   │   ├── start.handler.ts
│   │   │   ├── menu.handler.ts
│   │   │   └── help.handler.ts
│   │   ├── invoice/
│   │   │   ├── new.handler.ts      # /new — создание
│   │   │   ├── upload.handler.ts  # приём файла
│   │   │   ├── review.handler.ts  # ревью позиций
│   │   │   └── approve.handler.ts  # одобрение
│   │   ├── operator/
│   │   │   ├── list.handler.ts
│   │   │   ├── history.handler.ts
│   │   │   └── help-manager.handler.ts
│   │   ├── manager/
│   │   │   ├── team.handler.ts
│   │   │   ├── add-operator.handler.ts
│   │   │   ├── reassign.handler.ts
│   │   │   └── team-list.handler.ts
│   │   └── owner/
│   │       ├── admin.handler.ts
│   │       ├── add-manager.handler.ts
│   │       ├── billing.handler.ts
│   │       └── broadcast.handler.ts
│   ├── middleware/
│   │   ├── auth.middleware.ts      # проверка telegram_user_id в БД
│   │   ├── role.middleware.ts      # проверка роли
│   │   └── logging.middleware.ts
│   ├── fsm/                          # state machine
│   │   ├── states.enum.ts
│   │   └── transitions.ts
│   ├── services/
│   │   ├── api.service.ts          # клиент Sprint 2 API
│   │   ├── supabase.service.ts
│   │   ├── notifications.service.ts
│   │   └── files.service.ts
│   └── keyboards/                    # inline keyboards
│       ├── main-menu.kb.ts
│       ├── review.kb.ts
│       └── admin.kb.ts
```

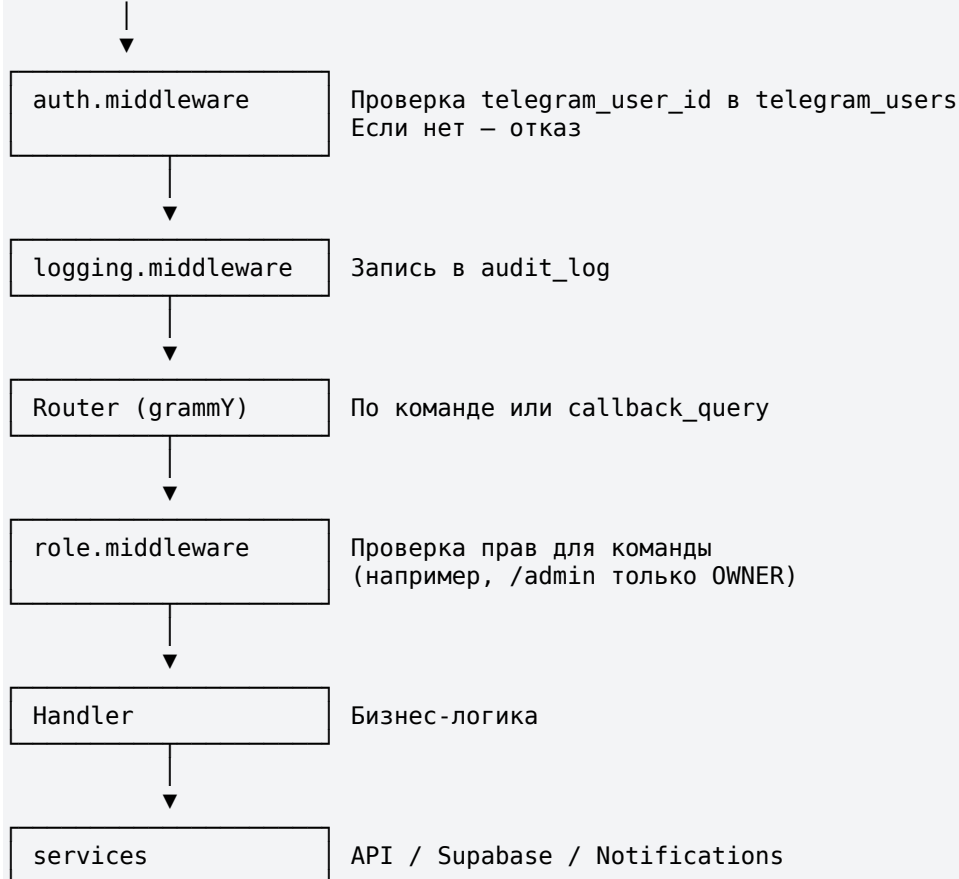
```

├── team.kb.ts
├── main.ts
├── tests/
└── package.json

```

6.3. Поток обработки сообщения

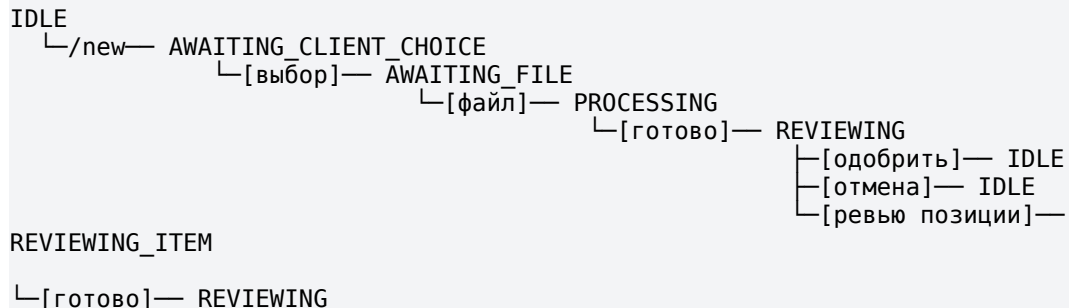
Сообщение от пользователя



6.4. Машина состояний (FSM)

Каждый пользователь находится в одном состоянии. Переходы между состояниями описаны явно. Без FSM невозможно правильно обработать многоэтапный диалог («загрузи файл» → «выбери клиента» → «выбери режим» → «введи сумму»).

Главный flow создания инвойса:



Команда `/cancel` из любого состояния → IDLE

7. Интеграция с API (Sprint 2)

Telegram-бот не содержит бизнес-логику классификации. Он только UX-слой. Вся логика — в API Sprint 2.

7.1. Используемые эндпоинты

Эндпоинт	Когда вызывается из бота
POST <code>/api/invoices/upload</code>	Пользователь отправил файл боту → бот загружает в API
POST <code>/api/invoices/:id/classify</code>	После выбора режима и суммы — запуск классификации
GET <code>/api/invoices/:id</code>	Опрос статуса (polling каждые 5 сек) или webhook
PATCH <code>/api/invoices/:id/items/:i</code>	Ручная правка позиции в ревью
POST <code>/api/invoices/:id/approve</code>	Кнопка «  Одобрить» → API генерит xlsx
GET <code>/api/clients</code>	Список доступных клиентов для оператора
GET <code>/api/tnved/search</code>	«Свой код» в ревью позиции (поиск по БД ТН ВЭД)

7.2. Идемпотентность

Каждое сообщение в Telegram имеет `update_id`. Бот сохраняет последний обработанный `update_id`, чтобы при перезапуске не обработать одно и то же дважды. Также при отправке HTTP-запросов в API используется `X-Idempotency-Key` заголовок.

7.3. Длинные операции (polling vs webhook)

Классификация занимает 2-3 минуты. Два варианта:

- Polling: бот опрашивает `GET /api/invoices/:id` каждые 5 секунд. Просто, но нагружает API
- Webhook (рекомендую): API вызывает POST на бот когда задача готова. Эффективнее, требует публичного URL

Решение: На MVP делаем polling (проще). После 1 месяца работы переключаем на webhook если будет нагрузка.

8. Технические детали

8.1. Хранение файлов

Когда пользователь отправляет packing list, Telegram даёт нам file_id. По file_id можно получить файл через Telegram Bot API.

```
// Алгоритм:
1. Получить file_id из сообщения
2. Вызвать bot.api.getFile(file_id) → file_path
3. Скачать файл по URL: https://api.telegram.org/file/bot{TOKEN}/{file_path}
4. Загрузить в Supabase Storage в bucket "uploads"
5. Передать URL Storage в API /api/invoices/upload

// Лимиты Telegram:
- Бот может скачивать файлы до 20 MB напрямую
- Для файлов > 20 MB нужен Local Bot API Server
- Обычный packing list китайцев — 1-5 MB, так что лимит не проблема
```

8.2. Кнопки (inline keyboard)

Все интерактивные элементы реализуются через inline_keyboard. Каждая кнопка имеет callback_data, по которой бот распознаёт действие.

```
// Пример кнопок ревью
{
  inline_keyboard: [
    [
      { text: "✅ ОДОБРИТЬ", callback_data: "invoice:approve:0034" },
      { text: "✎ РЕДАКТИРОВАТЬ", callback_data: "invoice:edit:0034" }
    ],
    [
      { text: "❌ ОТМЕНИТЬ", callback_data: "invoice:cancel:0034" }
    ]
  ]
}

// callback_data ограничен 64 байтами — использовать короткие ID
```

8.3. Отправка длинных сводок

Telegram ограничивает сообщения 4096 символами. Если сводка инвойса с 67 позициями длиннее — разбиваем на несколько сообщений или отправляем как файл.

8.4. Прогресс-индикаторы

Классификация 2-3 минуты — пользователь должен понимать что бот не завис. Используем edit_message для обновления статуса:

```
Бот: 🔄 Парсинг packing list... (1/3)
[через 30 сек, edit того же сообщения]
Бот: 🔄 Классификация позиций... (2/3) — 23/67 готово
[через 60 сек, edit того же сообщения]
Бот: 🔄 Генерация инвойса... (3/3)
[через 5 сек]
Бот: ✅ Готово!
```

8.5. Обработка ошибок

Что делать если API упал во время классификации:

- Бот ловит ошибку → пишет пользователю «Ошибка. Попробуй через минуту»
- В audit_log записывается ERROR с деталями
- OWNER получает уведомление о критических ошибках
- Инвойс остаётся в состоянии FAILED, можно перезапустить кнопкой « Повторить»

8.6. Безопасность

- Telegram-бот никогда не хранит токены/пароли в сообщениях
- Все callback_data подписываются HMAC чтобы нельзя было подделать
- При попытке OPERATOR вызвать /admin → отказ + запись в audit_log
- Резервные копии БД — минимум раз в сутки
- Все секреты — в Railway/Vercel env vars, не в коде

9. Acceptance Criteria (приёмка спринта)

9.1. Функциональные критерии

- ✓ Бот зарегистрирован в @BotFather, доступен под именем @tnved_bot
- ✓ OWNER может назначить MANAGER командой /add_manager
- ✓ MANAGER может добавить OPERATOR командой /add_operator
- ✓ OPERATOR может создать инвойс от загрузки файла до получения xlsx
- ✓ Полный flow создания инвойса занимает <5 минут
- ✓ Сгенерированный xlsx идентичен тому, что генерирует API напрямую
- ✓ MANAGER видит все инвойсы своих операторов через /team_list
- ✓ MANAGER может переназначить инвойс с одного оператора на другого
- ✓ OPERATOR может запросить помощь у MANAGER через /help_manager
- ✓ MANAGER получает уведомление о запросе помощи
- ✓ OWNER видит всех пользователей через /all_users
- ✓ OWNER может удалить любого пользователя
- ✓ При попытке OPERATOR использовать /admin — отказ
- ✓ Команда /cancel работает из любого состояния, возвращает в IDLE

9.2. Технические критерии

- ✓ Бот работает 24/7, аптайм >99% за неделю тестов
- ✓ При перезапуске бота незавершённые диалоги восстанавливаются из Redis
- ✓ Сообщения обрабатываются с задержкой <500ms (кроме классификации)
- ✓ При классификации показывается прогресс-индикатор
- ✓ Все действия пользователей логируются в audit_log
- ✓ Идемпотентность: один и тот же update_id не обрабатывается дважды
- ✓ Покрываемость тестами ≥60% (юнит-тесты на handlers, services, middleware)
- ✓ Документация README с инструкцией деплоя

9.3. Демо-сценарий приёмки

На приёмке разработчик показывает за 30 минут:

- С нуля: ты как OWNER добавляешь MANAGER (Руслан)
- Руслан добавляет 2 OPERATOR (Айгерим, Данияр)
- Айгерим создаёт инвойс LINEA TRANSIT с целью \$5000
- Бот возвращает результат с 2 позициями на ревью
- Айгерим запрашивает помощь у Руслана
- Руслан переназначает инвойс на Данияра
- Данияр одобряет инвойс, получает xlsx
- Открываем xlsx — проверяем что итог \$4 975 (≤\$5000)
- Ты как OWNER смотришь /all_users и видишь всех 3 сотрудников

10. Конфигурация и деплой

10.1. Переменные окружения

```
# Telegram
TELEGRAM_BOT_TOKEN=1234567890:ABCdefGhijKLM...
TELEGRAM_WEBHOOK_URL=https://bot.tnved.ai/webhook # для prod
TELEGRAM_WEBHOOK_SECRET=random-256-bit-hex

# API из Sprint 2
TNVED_API_URL=https://api.tnved.ai
TNVED_API_KEY=internal-service-key

# Supabase
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_SERVICE_KEY=eyJh...

# Redis (FSM состояний)
REDIS_URL=redis://...

# OWNER (первичная инициализация)
INITIAL_OWNER_TG_USER_ID=123456789
INITIAL_OWNER_USERNAME=@bahytzhan
INITIAL_OWNER_NAME=Бахитжан

# Логирование
LOG_LEVEL=info
SENTRY_DSN=https://... # опционально
```

10.2. Первичная инициализация

При первом запуске бота нужно создать OWNER в БД. Делается через миграцию или CLI:

```
npm run init-owner
> Введите telegram_user_id владельца: 123456789
> Введите username: @bahytzhan
> Введите имя: Бахитжан
✓ OWNER создан. Можешь писать боту /start
```

10.3. Хостинг

- Production: Railway.app (Pro tier ~\$20/мес)
- Альтернатива: Render.com, Fly.io
- Redis: Upstash (бесплатный tier до 10К команд/день — хватит на старте)
- Sentry для отлова ошибок (бесплатный tier)
- Healthcheck endpoint /health для мониторинга

10.4. CI/CD

```
# .github/workflows/deploy.yml
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - checkout
```

```
- npm ci  
- npm test  
- railway deploy
```

```
# Деплой за 2 минуты
```

11. Что после Sprint 3

Sprint 3 даёт работающий Telegram-бот для команды из 3-6 сотрудников. Дальнейшие спринты:

Спринт	Что делаем	Длительность	Что даст
Sprint 4	Web-кабинет cabinet.tnved.ai	3 недели	UI для удобной работы с большими инвойсами (30+ позиций), аналитика
Sprint 5	Биллинг, KPI, уведомления клиентам	2 недели	Готовый SaaS для коммерческого запуска
Sprint 6+	Whitelabel-боты под клиентов, мобильное приложение	по необходимости	Масштабирование на множество клиентов

После Sprint 3 бизнес может уже работать. Команда LINEA TRANSIT использует бота, ты собираешь обратную связь, готовишься к запуску web-кабинета.

12. Риски Sprint 3

Риск	Вероятность	Митигация
Telegram забанит бота за подозрительную активность	Низкая	Использовать @BotFather корректно, не рассылать спам, держать <30 сообщений/сек
Сотрудники не привыкнут к боту, будут просить «как раньше»	Средняя	Сделать обучение (15 мин видео-инструкция), назначить «чемпиона» который покажет коллегам
Большой packing list (200+ позиций) не уместится в одно сообщение	Высокая	Сводку отправлять как xlsx-файл, в чате только итоги
Сотрудник случайно одобрит инвойс с ошибкой	Средняя	Перед одобрением — обязательный ревью спорных позиций, в audit_log полная история
OWNER забудет назначить MANAGER, операторы окажутся «беспризорными»	Низкая	Бот запрещает добавлять OPERATOR без MANAGER. Только OWNER может быть «головой» цепочки
API из Sprint 2 будет тормозить на больших партиях	Средняя	Прогресс-индикатор каждые 30 сек, таймаут 10 минут, после — флаг для ручной обработки

13. Приёмка и оплата

Заказчик: Бахитжан

Длительность спринта: 1 неделя (5 рабочих дней)

Команда: 1 backend-разработчик уровня middle (TypeScript)

Этапы оплаты

- День 0: предоплата 30% при подписании ТЗ
- День 3: промежуточная демонстрация (бот авторизует, базовые команды)
- День 5: финальная демонстрация по сценарию из раздела 9.3
- День 5+1: оплата 70% после успешной приёмки

Документация

- README.md — инструкция запуска
- docs/architecture.md — диаграмма потоков
- docs/operator-guide.md — инструкция для сотрудников (русский, скриншоты)
- CHANGELOG.md
- .env.example

— Конец документа —